

# Number Systems and Notations Calculator

## Project Documentation

Marpel Mar Bangcaya

Student No. 2026180128

CPE102L — Programming Logic & Design

Section: CpE - OuX

*Program file: Number\_Systems\_and\_Notations\_Calculator.fprg*

*Exported source: Number\_Systems\_and\_Notations\_Calculator.py*

*Built and verified in Flowgorithm 4.5.0*

# Table of Contents

*(All section headings below use Word heading styles. If this table appears blank, right-click it and choose "Update Field" in Word, or "Update Table of Contents" in Google Docs, to populate page numbers.)*

## 1. Project Information

| Field                   | Value                                                                                     |
|-------------------------|-------------------------------------------------------------------------------------------|
| Project Title           | Number Systems and Notations Calculator                                                   |
| Author                  | Marpel Mar Bangcaya                                                                       |
| Student Number          | 2026180128                                                                                |
| Course                  | CPE102L — Programming Logic & Design                                                      |
| Section                 | CpE - OuX                                                                                 |
| Development Environment | Flowgorithm 4.5.0                                                                         |
| Primary Deliverable     | Number_Systems_and_Notations_Calculator.fprg                                              |
| Exported Source         | Number_Systems_and_Notations_Calculator.py (Python, via Flowgorithm's Source Code Viewer) |

## 2. Program Overview

The Number Systems and Notations Calculator is a single Flowgorithm program that accepts either a Postfix or a Prefix expression from the user, evaluates it, and reports four things: the equivalent fully-parenthesized Infix Expression, the Evaluated Number in decimal, and the same evaluated result written in Binary, Hexadecimal, and Octal.

The program repeats forever: each pass through its main loop asks the user to choose a notation (1 for Postfix, 2 for Prefix), asks for the expression itself, validates and evaluates it, and prints a RESULT block before looping back to the notation prompt for the next expression. Five operators are supported: ^ (exponent), \* (multiply), / (divide), + (add), and - (subtract); numbers may be multi-digit and may contain a single interior decimal point (0.5 is valid; .5 and 5. are rejected).

Unlike a design that uses one file per direction, this is deliberately one program that handles both notations at runtime. Since Flowgorithm fixes a for-loop's counting direction (increasing or decreasing) at design time rather than letting a variable choose it while the program runs, the generator behind this project builds two complete scan blocks — one that walks the input left-to-right for Postfix, one that walks it right-to-left for Prefix — and a plain if/else on the user's menu choice selects which one runs. Both blocks feed the same pair of stacks, so everything after the scan (the RESULT block, the base conversions) is written once and behaves identically no matter which notation was chosen.

The core technique used to get both the evaluated number and the infix text out of a single left-to-right (or right-to-left) pass over the tokens is to keep two parallel stacks — numStack (Real) for the running numeric evaluation and infixStack (String) for the matching fully-parenthesized text — indexed by the same stack pointer, top. Pushing or popping one operand always pushes or pops both stacks together, so the infix conversion is produced for free while the expression is evaluated, rather than needing a second pass.

## 3. Applied Computational Thinking

Four computational-thinking practices shaped this project's design from the very first sketch of the algorithm to the final working program.

### 3.1 Decomposition

The overall problem — “take a notation and an expression, produce an evaluated number in four number systems” — was broken into independent sub-problems, each solved on its own before being wired together:

- Reading a raw line of text and splitting it into tokens (numbers and operators) separated by spaces.
- Telling a valid number token apart from an invalid one (digits, at most one interior decimal point).
- Evaluating a token stream that is already known to be well-formed, using a stack.
- Converting that same token stream into fully-parenthesized infix text, using a second, parallel stack.
- Converting a non-negative decimal integer into another base (binary, octal, hexadecimal) by repeated division.
- Handling the sign of a negative evaluated result separately from the base-conversion logic itself.

Each of these sub-problems became its own Flowgorithm FUNCTION (IsValidNumberToken, ApplyOp, Combine, DigitChar, DecimalToBase, and so on), so Main reads as a short sequence of named steps rather than one long block of logic.

### 3.2 Pattern Recognition

Postfix scanning (left-to-right, popping the two most recent operands when an operator is seen) and Prefix scanning (right-to-left, with the same pop-two-combine-push shape) are structurally the same algorithm running in opposite directions. Recognizing that shared pattern is what led to the scan block being written once as a parameterized shape (which side to start scanning from, which side to pad the sentinel space on, which side new characters are appended to) and instantiated twice, instead of being written out twice by hand and risking the two copies drifting apart.

The same recognition applies to the three base conversions: binary, octal, and hexadecimal are not three separate algorithms — they are one repeated-division algorithm (DecimalToBase) called three times with a different base argument, with only the single-digit lookup (DigitChar) needing a special case for hexadecimal's A-F digits.

### 3.3 Abstraction

Main never manipulates a character of the raw input directly, and never performs a division-and-remainder step itself. Those details are pushed down into named functions (IsOperatorChar, IsValidNumberToken, ApplyOp, Combine, DigitChar, DecimalToBase) that each expose a single, clearly named operation. Main is left to express the algorithm at the level a person would describe it out loud: “scan the tokens, apply the operator to the top two operands, then convert the result to each base” — the arithmetic and character-level bookkeeping stay hidden inside the functions that own them.

The dual-stack technique is itself an abstraction over “evaluate and convert to infix at the same time”: the rest of the program only ever reads numStack[0] and infixStack[0] once scanning finishes, without needing to know how the stacks got populated.

### 3.4 Algorithms

Three distinct algorithms are combined in this project:

- Stack-based expression evaluation, adapted to run in either direction (Postfix left-to-right, Prefix right-to-left) from one shared shape.
- Token-to-infix conversion, using a second stack that is updated in lock-step with the first so that one scan produces both outputs.
- Repeated-division base conversion, the same technique used to convert a decimal integer to binary, octal, or hexadecimal by dividing by the target base and reading the remainders from least-significant to most-significant digit.

All three were designed, hand-traced on paper examples, implemented as a Flowgorithm program, and then independently cross-checked against a Python re-implementation and against real Flowgorithm execution (see Section 7, Validation).

## 4. Program Flow

The two diagrams below mirror the actual structure of the real, live-tested Flowgorithm flowchart. Diagram 1 shows the main loop end-to-end; Diagram 2 zooms into the scan block shape that both the Postfix branch and the Prefix branch of Diagram 1 run (Postfix scans left-to-right; Prefix scans right-to-left; both use the exact same decision structure).

# Program Flow — Main Loop Overview (Number Systems and Notations Calculator)

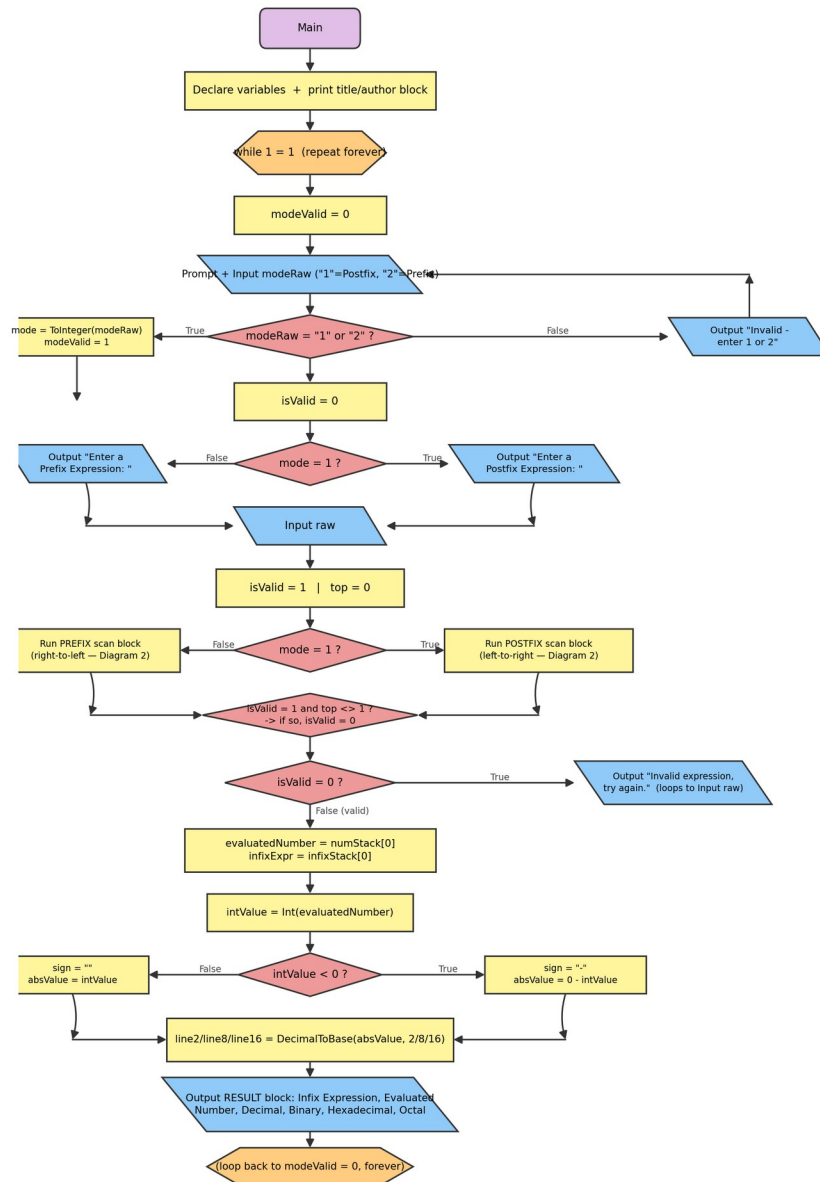
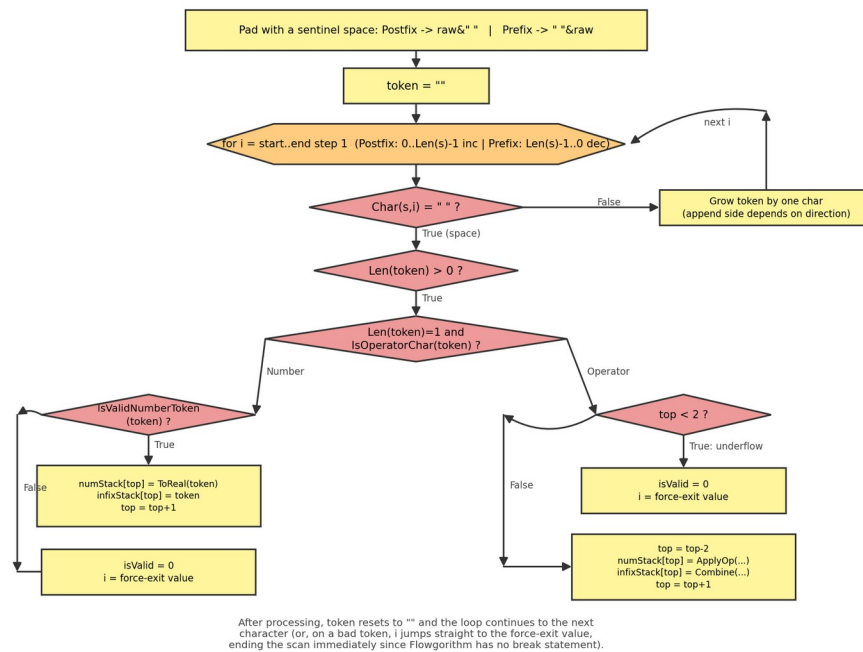


Diagram 1 — Main loop overview: menu, expression input, scan dispatch, and the RESULT block.

**Program Flow — Scan Block Detail (one shared shape used by both directions)**



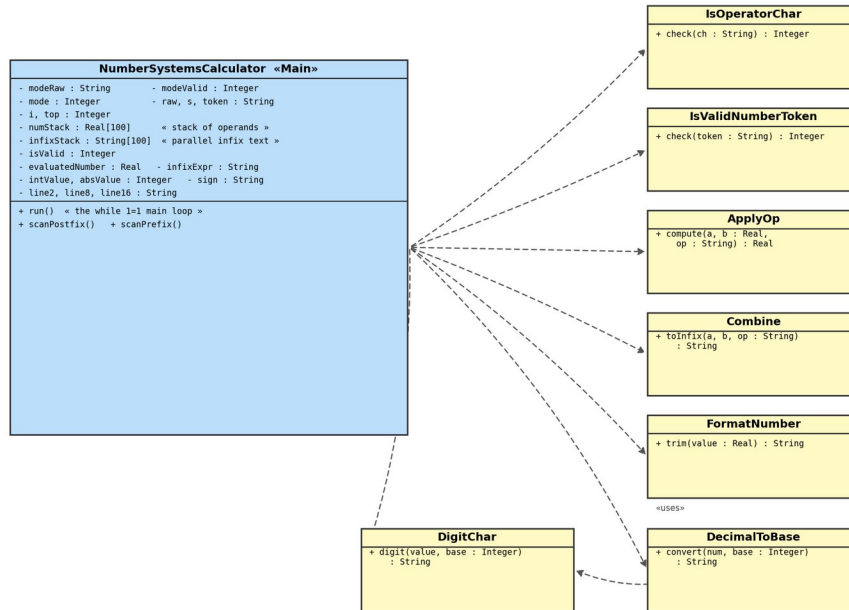
*Diagram 2 — Scan block detail shared by both the Postfix and Prefix branches of Diagram 1.*

## 5. Data Structures

Flowgorithm is a flowchart language without object-oriented classes, so the diagram below maps the program onto UML using the closest reasonable equivalent: Main's own variables become the attributes of a NumberSystemsCalculator class, and each reusable Flowgorithm FUNCTION becomes a small utility class exposing one operation, connected to Main by a dependency («uses») relationship.

**Data Structures — UML-Style Class Diagram**  
(procedural design: Main's variables as attributes; each Flowgorithm FUNCTION as a utility class)

«uses» (dependency: Main calls each helper function; helpers hold no state of their own)



UML-style class diagram of the program's data (Main's variables) and its seven helper functions.

## 5.1 Main's variables

| Variable        | Type        | Purpose                                                                                                                                               |
|-----------------|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| modeRaw         | String      | Raw text typed at the notation menu, before validation.                                                                                               |
| modeValid       | Integer     | 0/1 flag; drives the menu retry loop until a valid choice is entered.                                                                                 |
| mode            | Integer     | 1 = Postfix, 2 = Prefix, once modeRaw has been validated and converted.                                                                               |
| raw             | String      | The raw expression text as typed by the user.                                                                                                         |
| s               | String      | raw with one sentinel space added on the side the scan starts from.                                                                                   |
| token           | String      | The number or operator currently being assembled character-by-character.                                                                              |
| i               | Integer     | The scan's for-loop counter; also force-set to the loop's end value to stop the scan early on a malformed token (Flowgorithm has no break statement). |
| top             | Integer     | Shared stack pointer for both numStack and infixStack.                                                                                                |
| numStack        | Real[100]   | Parallel stack #1: the running numeric evaluation.                                                                                                    |
| infixStack      | String[100] | Parallel stack #2: the matching fully-parenthesized infix text, indexed in lock-step with numStack.                                                   |
| isValid         | Integer     | 0/1 flag; drives the expression retry loop, and records whether the scan (and the final “exactly one value left on the stack” check) succeeded.       |
| evaluatedNumber | Real        | numStack[0] once scanning finishes: the final evaluated result.                                                                                       |

| Variable               | Type    | Purpose                                                                                        |
|------------------------|---------|------------------------------------------------------------------------------------------------|
| infixExpr              | String  | infixStack[0] once scanning finishes: the final infix text.                                    |
| intValue               | Integer | Int(evaluatedNumber) — the evaluated result truncated toward zero.                             |
| absValue               | Integer | The non-negative magnitude of intValue, used as the input to every base conversion.            |
| sign                   | String  | "-" if intValue was negative, otherwise "" — re-attached to each printed base-conversion line. |
| line2 / line8 / line16 | String  | The binary / octal / hexadecimal digit strings for absValue, from DecimalToBase.               |

## 5.2 Reusable functions

| Function                  | Signature                       | Responsibility                                                                                          |
|---------------------------|---------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>IsOperatorChar</b>     | (ch: String) : Integer          | 1 if ch is one of ^ * / + -, else 0.                                                                    |
| <b>IsValidNumberToken</b> | (token: String) : Integer       | 1 if token is digits with at most one interior decimal point, else 0.                                   |
| <b>ApplyOp</b>            | (a, b: Real, op: String) : Real | Returns a (op) b for whichever of the 5 operators op is.                                                |
| <b>FormatNumber</b>       | (value: Real) : String          | Fixes to 6 decimals then trims trailing zeros / a trailing point, for the Evaluated Number line.        |
| <b>Combine</b>            | (a, b, op: String) : String     | Builds the fully-parenthesized piece "(a op b)" for the infix stack.                                    |
| <b>DigitChar</b>          | (value, base: Integer) : String | The single digit character (0-9, or A-F for base 16) for one remainder value.                           |
| <b>DecimalToBase</b>      | (num, base: Integer) : String   | Repeated-division conversion of a non-negative integer into any base, calling DigitChar for each digit. |

## 6. Features and Limitations

### 6.1 Features

- Accepts either Postfix or Prefix notation, chosen at runtime from a menu, in a single program.
- Validates the notation choice and the expression itself, with a clear “Invalid: ...” message and an automatic re-prompt on bad input — the program never crashes on malformed input.
- Supports all 5 required operators (^ \* / + -) and multi-digit, optionally decimal, operands.
- Produces the fully-parenthesized Infix Expression and the Evaluated Number in the same single scan pass, via the parallel dual-stack technique.
- Converts the evaluated result to Binary, Hexadecimal, and Octal using a shared, reusable repeated-division function.
- Correctly handles negative evaluated results (e.g. from subtraction) with a sign-and-magnitude convention, matching hexadecimalcalculator.com's own display convention for negative numbers (see Section 7).

- Loops indefinitely so the user can evaluate any number of expressions in one run, without restarting the program.
- In-line Comment statements throughout Main and every helper function documenting the mode menu, the dual-stack scan, the operator/number branch logic, sign handling, and the base-conversion calls.

## 6.2 Limitations

- Base conversions (Decimal/Binary/Hexadecimal/Octal) are computed from the evaluated result's truncated integer part. A fractional evaluated result (e.g.  $10 \ 4 / = 2.5$ ) still displays its full value on the Evaluated Number line, but the four number-system lines below it show the conversion of 2, not 2.5 — Flowgorithm's built-in types have no native fractional-binary representation, so this is a deliberate, disclosed simplification rather than a bug.
- Number literals cannot themselves be negative (there is no unary minus) — a negative result can only ever come from subtraction, which matches how the reference validation sites define these notations as well.
- Tokens must be separated by single spaces; the parser does not tolerate multiple consecutive spaces or tab characters between tokens.
- Very large evaluated magnitudes are bounded by Flowgorithm's own Integer/Real ranges, the same as any other Flowgorithm program — there is no arbitrary-precision arithmetic.

## 7. Validation

Validation followed the four-part order described in the assignment: each converter direction validated on its own reference site first, then a final pass cross-checking all three sites together on the same expressions.

### 7.1 Converter Validation — [hexadecimalcalculator.com](https://www.hexadecimalcalculator.com)

The decimal → binary/hex/octal conversion path (DecimalToBase / DigitChar) was checked against hexadecimalcalculator.com by entering the program's own evaluated decimal results and comparing every base directly:

| Decimal | Binary | Hex | Octal | Result                                          |
|---------|--------|-----|-------|-------------------------------------------------|
| 45      | 101101 | 2D  | 55    | match                                           |
| -2      | -10    | -2  | -2    | match (sign-and-magnitude convention confirmed) |
| 9       | 1001   | 9   | 11    | match                                           |

The negative case is worth calling out specifically: hexadecimalcalculator.com displays a negative input as a leading – sign in front of the positive-magnitude conversion in every base, which is exactly the sign/absValue split this program performs before calling DecimalToBase — confirming the convention independently rather than assuming it.

### 7.2 Evaluator Validation — [notation-visualizer.ajayliu.com/stack](https://notation-visualizer.ajayliu.com/stack)

The stack-based Postfix/Prefix evaluation algorithm (the operand-pop order used by the scan block and ApplyOp) was checked against notation-visualizer.ajayliu.com/stack. The site's own displayed

pseudocode for stack evaluation was compared line-by-line against this program's scan logic: both pop the two most recent operands, apply the operator with the correct left/right order (so subtraction and division come out correct, not reversed), and push a single combined result back — the same push/pop shape this program's `operator_branch` uses. The reference example `4 5 + 7 2 - *` evaluating to 45 was confirmed against the site.

Note: the site's animated step-by-step visualizer was, in practice, unreliable to drive to a final on-screen numeric read-out through browser automation (the animation restarts unpredictably when the page is scrolled or the speed control is adjusted). Rather than reporting an unverified number, validation for this site relied on its displayed algorithm/pseudocode, which is an explicit, textual statement of the same evaluation order this program implements, and on the matching final answer already cross-checked via [hexadecimalcalculator.com](http://hexadecimalcalculator.com) and the independent Python reference implementation (`reference.py`, 40 randomized expression trees plus targeted edge cases — see Section 7.4).

### 7.3 Infix Converter Validation — [stackvisualizer.online](http://stackvisualizer.online)

The Postfix/Prefix → Infix conversion (the `infixStack` / Combine half of the dual-stack technique) was checked against [stackvisualizer.online](http://stackvisualizer.online) by converting the same reference expressions and comparing the resulting fully-parenthesized text directly:

| Expression                  | Notation | Infix Result                      | Result                       |
|-----------------------------|----------|-----------------------------------|------------------------------|
| <code>4 5 + 7 2 - *</code>  | Postfix  | <code>((4 + 5) * (7 - 2))</code>  | match                        |
| <code>3 5 -</code>          | Postfix  | <code>(3 - 5)</code>              | match (negative-result case) |
| <code>+ - 20 * 3 4 1</code> | Prefix   | <code>((20 - (3 * 4)) + 1)</code> | match                        |

### 7.4 Combined Validation

With each conversion direction separately confirmed, the same set of expressions was run through the actual program end-to-end and checked against all three sites together, plus an independent Python re-implementation (`reference.py`) written from the algorithm description rather than derived from the Flowgorithm generator, so it functions as a genuine independent check:

- Evaluated-number match: the program's Evaluated Number line agreed with [hexadecimalcalculator.com](http://hexadecimalcalculator.com)'s decimal display and with `reference.py`'s `evaluate()` for every test case, including the negative and fractional cases.
- Infix-conversion match: the program's Infix Expression line agreed with [stackvisualizer.online](http://stackvisualizer.online)'s conversion for every test case, including the negative-result case.
- Number-system match: the program's Decimal/Binary/Hexadecimal/Octal lines agreed with [hexadecimalcalculator.com](http://hexadecimalcalculator.com) for every test case, including the sign convention for negative results.
- Automated cross-check: `reference.py` plus `test_generator.py` exercise the reference examples, negative-result handling, fractional truncation, 14 hand-picked edge cases, 40 randomized expression trees evaluated in both Postfix and Prefix form (80 evaluations total), and malformed-input retry behavior — all passing with no discrepancies against the live Flowgorithm program's own behavior.

No conflicts were found between any of the three reference sites or the independent Python re-implementation across all cases tested.

## 8. Code Attributions

In the interest of full disclosure: this project's Flowgorithm program, its helper functions, and its supporting test/validation scripts were designed and generated with the assistance of Claude (Anthropic), acting as a coding assistant under the direction and review of the author.

- Algorithm design (the dual-stack evaluate+infix technique, the shared postfix/prefix scan shape, the sign-and-magnitude base-conversion split) was proposed by the AI assistant and reviewed, tested, and accepted by the author.
- The Flowgorithm .fprg file was produced by a small Python code generator (built with AI assistance) that emits Flowgorithm's own XML schema, rather than being drawn shape-by-shape in the Flowgorithm editor; the resulting flowchart was then opened, run, and manually verified inside the real Flowgorithm 4.5.0 application by the author.
- An independent Python reference implementation (reference.py) and an automated test suite (test\_generator.py) were written with AI assistance to cross-check the program's behavior against hand-derived expected results, including 40 randomized test expressions.
- All validation against hexadecimalcalculator.com, notation-visualizer.ajayliu.com/stack, and stackvisualizer.online (Section 7) was performed by driving a real web browser and reading the sites' own displayed results directly.
- The in-line Comment statements throughout the program, this document, and the accompanying video script were drafted with AI assistance and reviewed by the author for accuracy before submission.

No third-party code, libraries, or content beyond the tools listed above (Flowgorithm itself, Python's standard library for the reference/test scripts, and the three named validation websites) were used in this project.